

comp10002 Foundations of Algorithms

Semester Two, 2025

Why Algorithms?!

© The University of Melbourne, 2025
Lecture slides prepared by Alistair Moffat

A Problem

Method 0

Method 1

Method 2

Homework

A simple computational problem

Simply a matter of programming

Making it go faster

Adding algorithmic design

Homework

Given: A sequence T of n symbols

Problem: Find all pairs of locations in T at which repeated subsequences of length m arise.

Example

Try

`fat.rat.eat.bat.cat.eat.fat.rat.`

with $m = 7$?

A Problem

Method 0

Method 1

Method 2

Homework

Try

```
fat.rat.eat.bat.cat.eat.fat.rat.
```

with $m = 7$?

```
mac: findrepeats0 7 < test1.txt
n = 32
m = 7
0 24: fat.rat
1 25: at.rat.
5 17: at.eat.
count = 3
```

[A Problem](#)

[Method 0](#)

[Method 1](#)

[Method 2](#)

[Homework](#)

Method 0: Just simple programming, right?

```
read the sequence  $T$ 
for  $i = 0$  to  $n - m$  do
  for  $j = i + 1$  to  $n - m$  do
    // compare the  $m$ -character strings starting at positions  $i$  and  $j$ 
    for  $k = 0$  to  $m - 1$  do
      compare  $T[i + k]$  against  $T[j + k]$ 
    if all  $m$  characters were the same then
      write an output line
```

Works just fine.

[A Problem](#)

[Method 0](#)

[Method 1](#)

[Method 2](#)

[Homework](#)

Method 0: Just simple programming, right?

```
read the sequence  $T$ 
for  $i = 0$  to  $n - m$  do
  for  $j = i + 1$  to  $n - m$  do
    // compare the  $m$ -character strings starting at positions  $i$  and  $j$ 
    for  $k = 0$  to  $m - 1$  do
      compare  $T[i + k]$  against  $T[j + k]$ 
    if all  $m$  characters were the same then
      write an output line
```

Works just fine.

Until you give it non-trivial data.

[A Problem](#)

[Method 0](#)

[Method 1](#)

[Method 2](#)

[Homework](#)

Huh? Why so slow??

Look at those loops! To execute the program, takes
 $n \times n \times m = n^2 m$ individual character comparisons.

Important fact: computers do round a billion, or 10^9 , elemental operations a second. When $n = 1,000$ and $m = 10$, gives $n^2 m = 10^7$ operations, or maybe 0.01 seconds.

Huh? Why so slow??

Look at those loops! To execute the program, takes $n \times n \times m = n^2 m$ individual character comparisons.

Important fact: computers do round a billion, or 10^9 , elemental operations a second. When $n = 1,000$ and $m = 10$, gives $n^2 m = 10^7$ operations, or maybe 0.01 seconds.

When $n = 1,000,000$ and $m = 100$, there will be $n^2 m = 10^{14}$ operations, taking $\approx 10^5$ seconds.

That's a whole day!

Method 1: Easy to fix...

Why test the full m characters every time, can surely do less work *on average* than that?

read the sequence T

for $i = 0$ **to** $n - m$ **do**

for $j = i + 1$ **to** $n - m$ **do**

 compare up to m characters starting at positions i and j ,
 stopping as soon as any discrepancy is found

if no discrepancy was found **then**

 write an output line

Method 1: Easy to fix... ???

This makes it faster (for *average* input).

Worst case is still n^2m operations (what kind of text?)

On average, each check fails very early; now spend kn^2 operations for a text of n characters, where $k \ll m$ is (hopefully) a small constant a little bigger than 1.0.

[A Problem](#)

[Method 0](#)

[Method 1](#)

[Method 2](#)

[Homework](#)

Method 1: Easy to fix... ???

This makes it faster (for *average* input).

Worst case is still n^2m operations (what kind of text?)

On average, each check fails very early; now spend kn^2 operations for a text of n characters, where $k \ll m$ is (hopefully) a small constant a little bigger than 1.0.

Faster, but still not fast enough to be “scalable”.

For $n = 1,000,000$ gives rise to (at least) 10^{12} operations. And that will still be something like a thousand seconds, or 17 minutes.

[A Problem](#)

[Method 0](#)

[Method 1](#)

[Method 2](#)

[Homework](#)

Method 2: Here comes an algorithm!

Algorithms are the aspect of computing that require **science**.

There will be a variety of ways of achieving any given goal, with subtle and not-so-subtle differences between them.

It is often possible to trade memory space for execution time; and exploit selective pre-computation.

Sophisticated **data structures** are chosen to match the exact mix of operations that need to be supported.

Method 2: Here comes an algorithm!

```
read the sequence  $T$ 
for  $i = 0$  to  $n - m$  do
     $S[i] \leftarrow i$ 
sort  $S[i]$  based on  $T[S[i]], T[S[i] + 1], \dots$ 
for  $i = 0$  to  $n - m$  do
    compare at most  $m$  characters starting at
         $T[S[i]]$  and  $T[S[i + 1]]$ 
    if no differences are found then
        write an output line
```

Array $S[]$ is a [suffix array](#). This sketch finds one instance of each repetition; an easy modification finds all of them.

[A Problem](#)

[Method 0](#)

[Method 1](#)

[Method 2](#)

[Homework](#)

Method 2: Execution time?

Sorting n items takes fewer than $2n \log n$ comparisons.

In a simple implementation, each step is a string comparison over (still never more than) m characters.

So for $n = 1,000,000$ and $m = 100$ now have fewer than $\approx 2mn \log n = 4 \times 10^9$ operations.

Method 2: Execution time?

Sorting n items takes fewer than $2n \log n$ comparisons.

In a simple implementation, each step is a string comparison over (still never more than) m characters.

So for $n = 1,000,000$ and $m = 100$ now have fewer than $\approx 2mn \log n = 4 \times 10^9$ operations.

That's at most a few seconds, even in the worst case!

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s
(yes, that's $100\times$ more using $10\times$ less)			

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s
(yes, <i>that's</i> $100\times$ more using $10\times$ less)			
$n = 1$ GB			

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s
(yes, that's $100\times$ more using $10\times$ less)			
$n = 1$ GB		4.8c	

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s
(yes, that's $100\times$ more using $10\times$ less)			
$n = 1$ GB	4.8c	12y	

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s
(yes, that's $100\times$ more using $10\times$ less)			
$n = 1$ GB	4.8c	12y	7m

One prepared earlier...

Measured on 2024 Macbook Pro, M3 Pro chip (4.05 GHz) 36 GB memory, July 21 2024; with $m = 100$ and using WSJ; black is measured, red is estimated.

Method	0	1	2
$n = 10$ kB	1.50s	0.05s	0.01s
$n = 100$ kB	150s	3.3s	0.02s
$n = 1$ MB		394s	0.27s
$n = 10$ MB			2.8s
$n = 100$ MB			40s
(yes, that's $100\times$ more using $10\times$ less)			
$n = 1$ GB	4.8c	12y	7m

c = centuries, 100 years!

Method 2 can be improved if a tailored sorting mechanism is used.

Using a [ternary Quicksort](#) (a couple of hours of implementation effort), [Method 3](#) runs around 50% faster, and takes 22 seconds for 100 MB.

Using an explicit method for building a suffix array (perhaps a couple of days of implementation effort) and careful hand tuning might give rise to another halving.

Are we there yet???

No, not really.

The problem with Methods 2 and 3 is that sorting requires random access into the underlying string. That, in turn, means that we are limited by the available memory.

A 10 GB string would not be possible on the Macbook Pro.
Hmmm!

Method 4 uses more (disk) space, but processing can be parallelized over a **cluster** of computers. Setting up a parallel computation requires further deep insights into the problem structure.

Only a little more complex, and executes fast enough: with $n = 100$ MB and $m = 100$, requires 21 seconds; with estimated time for $n = 1$ GB of 4 minutes, and for $n = 1$ TB of 3.5 days.

Or, around 5 minutes for $n = 1$ TB when spread over 1,000 processors in a data center.

Method 4 uses more (disk) space, but processing can be parallelized over a **cluster** of computers. Setting up a parallel computation requires further deep insights into the problem structure.

Only a little more complex, and executes fast enough: with $n = 100$ MB and $m = 100$, requires 21 seconds; with estimated time for $n = 1$ GB of 4 minutes, and for $n = 1$ TB of 3.5 days.

Or, around 5 minutes for $n = 1$ TB when spread over 1,000 processors in a data center.

Method 4 is an **invention**!

What about Python then?

Couldn't we just use a Python dictionary?

```
i, j, d = 0, m, {}  
while j ≤ n do  
    d[T[i : j]].append(i)  
    i, j = i + 1, j + 1  
for s in d do  
    if len(d[s]) > 1 then  
        print(d[s])
```

Yes, could. When $n = 10$ MB the execution time is 10s, (only) $6\times$ longer than Method 3. *But more than 4 GB of memory is required.*

More importantly, [how does the dictionary get implemented?](#) (Fun fact: most Python implementations are partially written in C!)

Given: A sequence T of n symbols.

Problem: Determine if there is any value in T that occurs more than $n/2$ times, and if so, what the value is.

Target: *Worst case linear time, constant additional space.*

As algorithmic researchers, we like nothing more than:

- ▶ Real data, and a precise description of the transformation or processing that must be performed against it; plus
- ▶ An understanding of what the end application/benefit will be from the desired techniques; plus
- ▶ Smart colleagues and PhD students to work with us.

Algorithms are Fun!